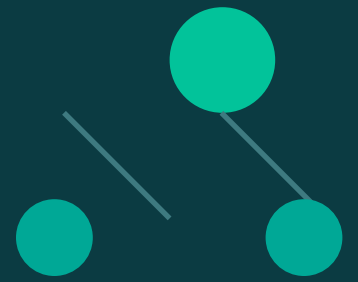
MACHINE LEARNING · CLASSROOM MODULE

CART

Classification And Regression Trees

How decision trees learn to split data, make predictions, and avoid overfitting



Instructor Notes | Source: GeeksforGeeks — updated 4 Dec 2025

CLASS ROADMAP

Today's Learning Objectives

01

Understand what CART is

Why it's the foundation of Random Forest, Gradient Boosting and XGBoost

02

Learn how CART builds a tree

The 4-step recursive splitting process

03

Compare classification vs. regression

Gini Impurity vs. RSS / MSE splitting criteria

04

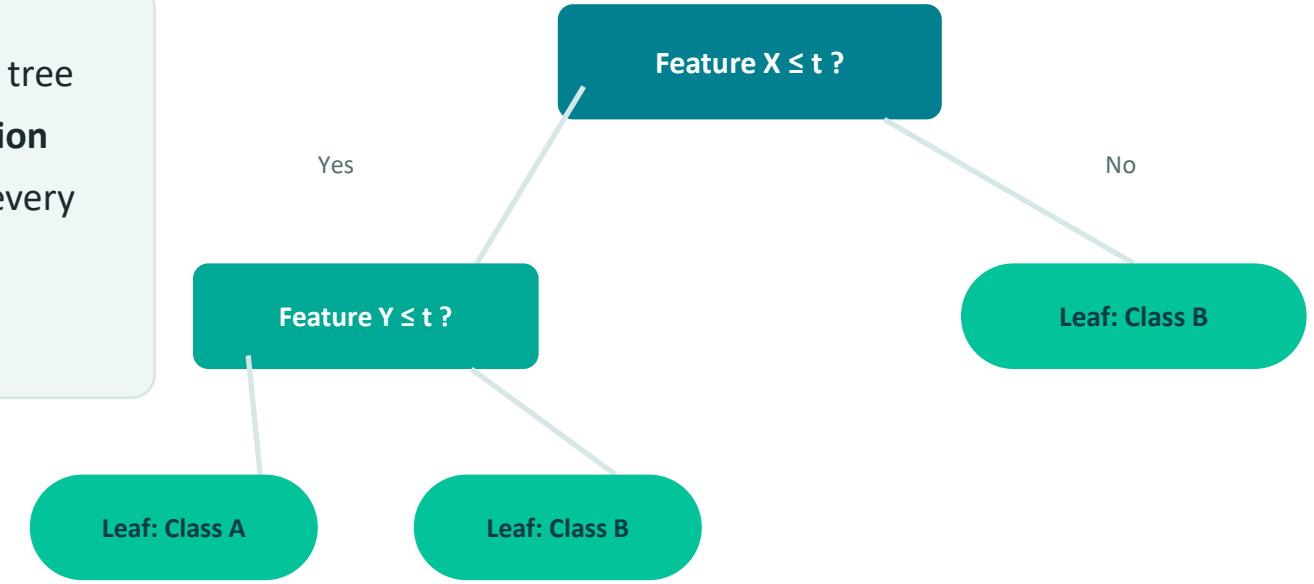
Explore pruning & hyperparameters

How CART avoids overfitting and stays generalizable

THE BASICS

What is CART?

CART (Classification And Regression Tree) builds a decision tree that **predicts outcomes for both classification and regression tasks** by splitting data based on rules that reduce error at every step.



Every internal node asks a yes/no question; every leaf is a final prediction.

- Uses binary splits at every node
- Simple to understand and interpret
- Works for both numeric and categorical data
- Forms the base of advanced models like Random Forest, Gradient Boosting and XGBoost

THE CORE IDEA

How CART Builds a Decision Tree

CART constructs a decision tree by recursively splitting the dataset based on the feature and threshold that produce the highest reduction in impurity (classification) or error (regression).

1

Evaluate Best Split

Check every feature & threshold; compute impurity or error

2

Select Optimal Split

Compare reductions; pick the best feature-threshold pair

3

Create Binary Nodes

Split into Left (≤ t) and Right (> t) child nodes

4

Recurse

Repeat on each child until a stopping condition is met

STEPS 1 – 2

Finding the Best Split

1 Evaluate Best Split for Each Feature

- Check all features and their possible split values
- Compute impurity or error for each candidate
- Identify the split with the maximum reduction in impurity or error

2 Select the Optimal Split

- Compare the impurity decrease across all features
- Choose the feature and threshold pair with the best score
- This becomes the splitting rule for the current node

STEPS 3 – 4

Building the Tree Structure

3 Create Binary Child Nodes

- Split into Left Node (values ≤ threshold)
- and Right Node (values > threshold)
- Assign corresponding samples to each child node

4 Apply Recursive Splitting

- Repeat the same process for each child node
- Continue until a stopping condition is met
- The result is a hierarchy of binary decisions

CATEGORICAL OUTPUT

CART for Classification

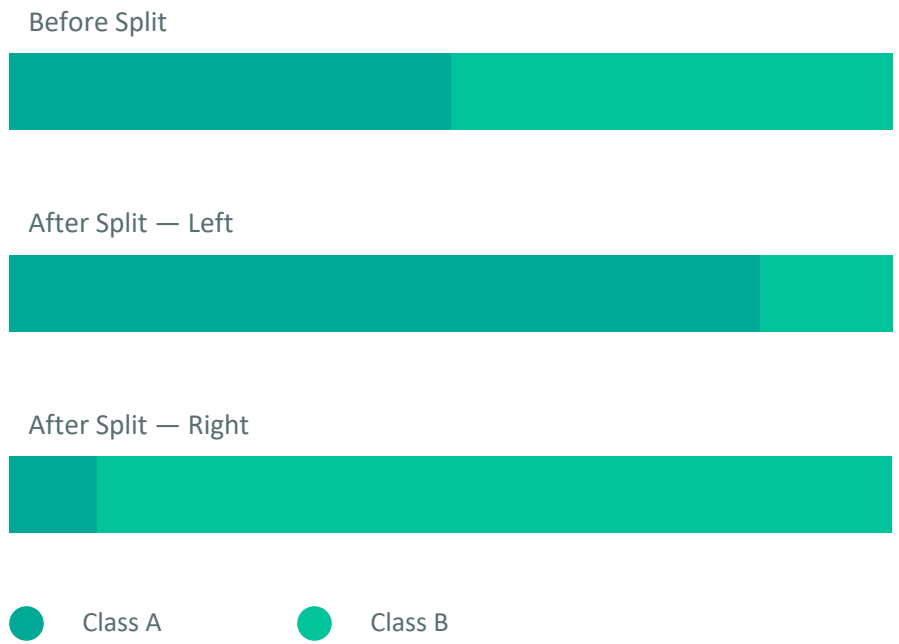
Used when the output variable is categorical.

How it Works

- Recursively splits the dataset to increase class purity at each step
- Uses Gini Impurity to select the best feature and threshold for splitting
- Continues splitting until a stopping rule is met — e.g. maximum tree depth or minimum required samples

CART chooses splits that produce the purest possible child nodes.

Node Purity Example



GINI IMPURITY

Splitting Criteria — Classification

CART uses Gini Impurity to measure how mixed the classes are in a node — how likely a randomly chosen sample would be misclassified if labeled according to the node's class distribution.

$$\text{Gini} = 1 - \sum p_i^2$$

p_i proportion of class i in the node

n number of classes

Gini = 0

Node is completely pure (only one class)

Gini → 1

Node contains a mix of many classes and is highly impure

Worked example (2 classes, 60/40 split):

$$\text{Gini} = 1 - (0.6^2 + 0.4^2) = 1 - 0.52 = 0.48$$

NUMERICAL OUTPUT

CART for Regression

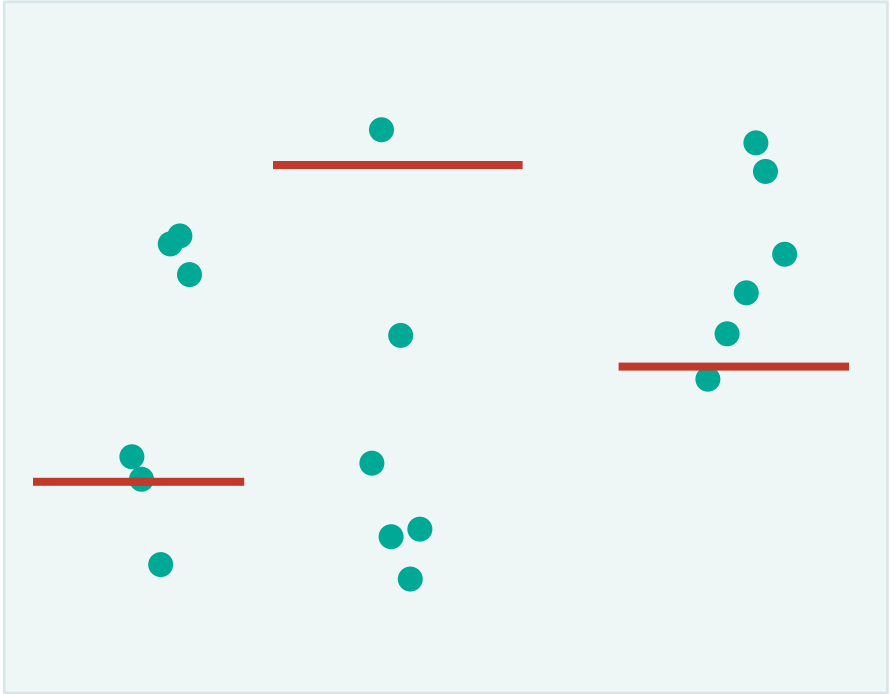
Used when the output variable is numerical.

How it Works

- Splits the data to minimize residual error between actual and predicted values
- Uses Mean Squared Error (MSE) or Residual Sum of Squares (RSS) as the splitting criterion
- Each leaf node stores the mean of the target values within it — this becomes the prediction for new samples

CART builds the tree that yields the lowest possible prediction error.

Leaf = Mean of Target Values



Red lines = predicted mean per leaf region

Splitting Criteria — Regression

For regression problems, CART uses Residual Sum of Squares (RSS) or Mean Squared Error (MSE) to find the best split. Lower RSS / MSE indicates a better split.

Residual Sum of Squares (RSS)

Measures the total squared difference between actual output values and predicted values.

$$\mathbf{RSS} = \Sigma (y - \hat{y})^2$$

y = actual value *ŷ = predicted value*

Sums error across ALL samples in the node — total error.

Mean Squared Error (MSE)

RSS divided by the number of samples — the average squared error per sample.

$$\mathbf{MSE} = 1/n \ \Sigma (y - \hat{y})^2$$

n = number of samples in the node

Normalized error — comparable across nodes of different sizes.

PREVENTING OVERFITTING

Pruning in CART

Pruning trims branches that add little or no improvement to model accuracy — simplifying the tree, improving generalization, and reducing model complexity.

1 Cost Complexity Pruning

Removes branches by comparing the trade-off between tree accuracy and tree size, keeping only nodes that significantly improve performance.

2 Reduced Error Pruning

Eliminates nodes that do not improve the model's accuracy on a validation dataset, ensuring only beneficial splits remain.

Common Stopping / Pruning Criteria

- **Max tree depth reached**
- **Too few samples to split**
- **No further impurity gain**
- **Node becomes pure**

Hyperparameters in CART

CART provides several hyperparameters to control tree structure, prevent overfitting, and improve model performance.

`max_depth`	Controls the maximum depth of the tree, preventing it from growing too deep and overfitting the data.
`min_samples_split`	Minimum number of samples required to split an internal node; higher values make the tree more conservative.
`min_samples_leaf`	Minimum number of samples allowed in a leaf node, helping avoid very small or unstable leaves.
`max_features`	Number of features considered while looking for the best split; reduces computation and variance.
`criterion`	Function used to evaluate a split — Gini or Entropy for classification, MSE for regression.

APPLYING CART · WORKED EXAMPLE

The Play Tennis Problem

Let's use CART, step by step, to build a real decision tree from data.

The Problem

Given 14 days of weather observations, predict whether tennis will be played — a classification task with a categorical (Yes / No) output.

Since the output is categorical, CART will use Gini Impurity as the splitting criterion at every node.

Input Features

Outlook
Sunny / Overcast / Rain

Temperature
Hot / Mild / Cool

Humidity
High / Normal

Wind
Weak / Strong

Target / Output Label: Play Tennis → Yes or No

WORKED EXAMPLE

The Play Tennis Dataset

14 observations · 4 input features · 1 categorical target (Play Tennis)

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

WORKED EXAMPLE

Step 1 — Gini of the Root Node

Out of 14 days, 9 resulted in Play = Yes and 5 in Play = No. This is the impurity we start with, before any split.

$$\text{Gini(root)} = 1 - (9/14)^2 - (5/14)^2$$

$$= 1 - 0.413 - 0.128$$

$$= 0.459$$

Reading it: 0.459 is fairly impure — the classes are reasonably mixed (close to a 50/50 split would give Gini ≈ 0.5).

Next: try splitting on each feature and see which one reduces this impurity the most.

Example Calculation — Splitting on Outlook

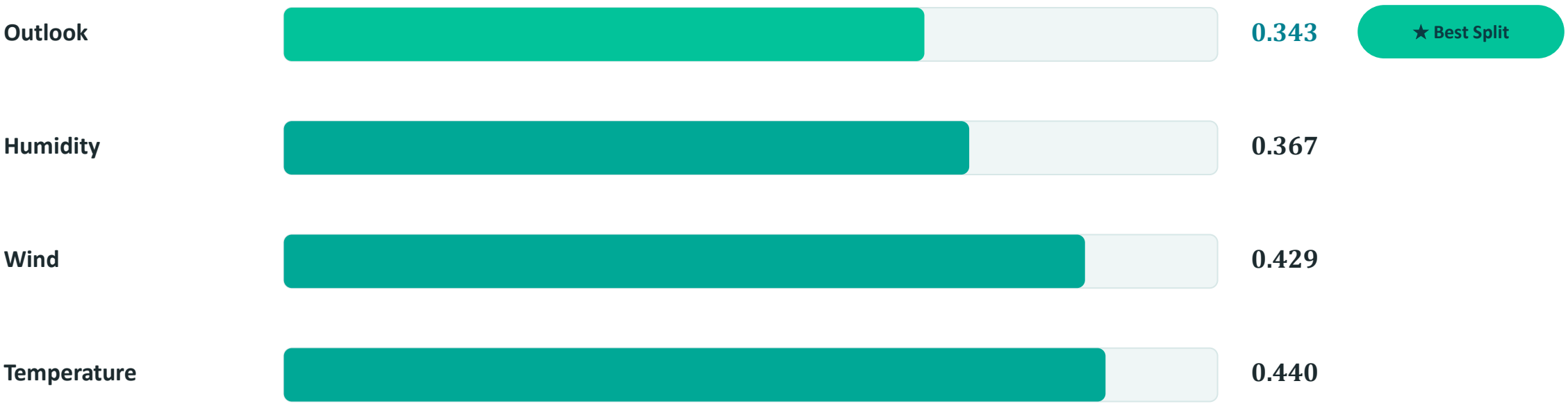
Sunny	2 Yes / 3 No	$1 - 0.16 - 0.36 = 0.48$
Overcast	4 Yes / 0 No	$1 - 1.00 - 0.00 = 0.00$
Rain	3 Yes / 2 No	$1 - 0.36 - 0.16 = 0.48$

$$\text{Weighted Gini(Outlook)} = (5/14)(0.48) + (4/14)(0) + (5/14)(0.48) = 0.343$$

WORKED EXAMPLE

Step 2 — Compare Every Feature

Repeat the weighted-Gini calculation for each feature, then pick the lowest score — the biggest drop in impurity.



Outlook produces the lowest weighted Gini (0.343) — CART selects Outlook as the root split.

WORKED EXAMPLE

Root Split on Outlook



Overcast is already pure (all Yes) — it becomes a leaf immediately. No further split needed.

Sunny and Rain are still mixed — CART recurses on each of these subsets independently.

Next: apply Steps 1–4 again, separately, inside the Sunny and Rain subsets.

WORKED EXAMPLE

Recursing on the Sunny Branch

Sunny subset (5 days):

```
Hot/High/Weak → No
Hot/High/Strong → No
Mild/High/Weak → No
Cool/Normal/Weak → Yes
Mild/Normal/Strong → Yes
```

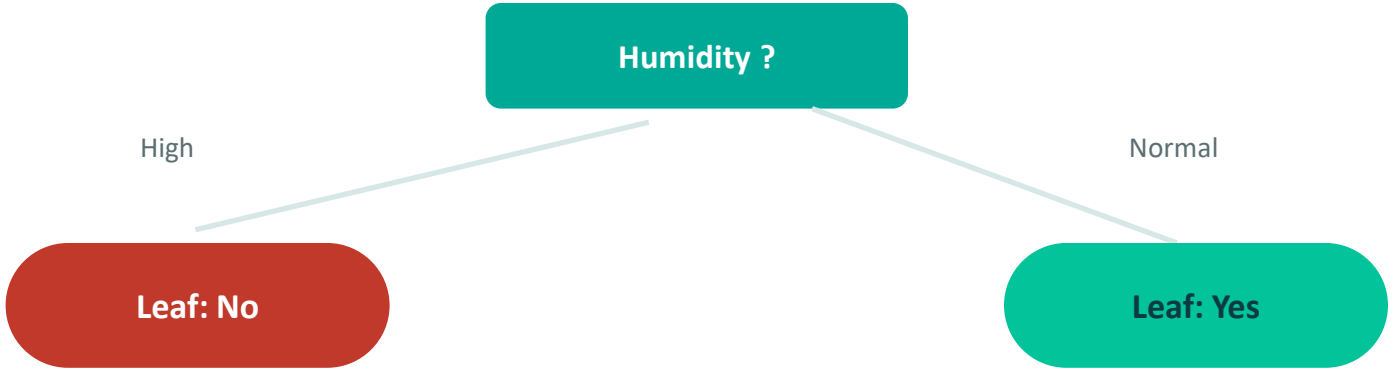
Testing Humidity within Sunny:

High (3 days): 0 Yes / 3 No → Gini = 0

Normal (2 days): 2 Yes / 0 No → Gini = 0

Weighted Gini(Humidity) = 0

A perfect split — no need to check Temperature or Wind.



WORKED EXAMPLE

Recursing on the Rain Branch

Rain subset (5 days):

```
Mild/High/Weak → Yes
Cool/Normal/Weak → Yes
Cool/Normal/Strong → No
Mild/Normal/Weak → Yes
Mild/High/Strong → No
```

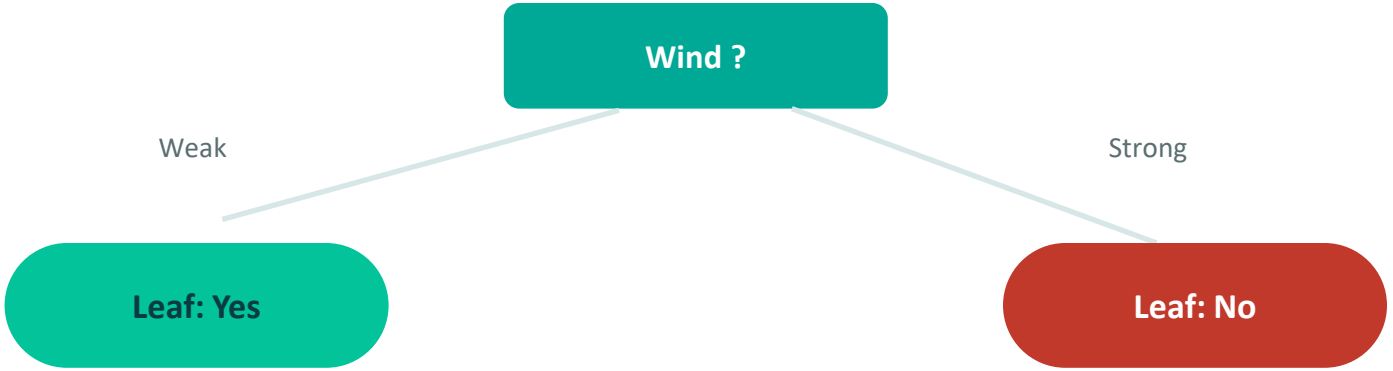
Testing Wind within Rain:

Weak (3 days): 3 Yes / 0 No → Gini = 0

Strong (2 days): 0 Yes / 2 No → Gini = 0

Weighted Gini(Wind) = 0

Another perfect split — Wind fully separates Yes from No.



WORKED EXAMPLE

The Final Decision Tree



Every path from root to leaf is a rule. e.g. Outlook = Rain AND Wind = Weak → Play = Yes

WORKED EXAMPLE

Using the Tree to Predict

New Day: Outlook = Sunny, Humidity = Normal

1. **Start at root: Outlook ?**

   Value = Sunny → go left

2. **At Sunny node: Humidity ?**

   Value = Normal → go right

3. **Reached a leaf**

   Leaf says: Yes

Prediction: Play Tennis = Yes

What This Example Taught Us

- Gini Impurity is computed at every node, not just the root
- CART always picks the feature with the lowest weighted Gini
- A branch stops splitting once it becomes pure (Gini = 0)
- Different branches can split on different features
- The resulting tree is just a set of nested yes/no rules

```
cart = DecisionTreeClassifier(
    criterion='gini',        # Splitting criterion
    max_depth=3,             # Maximum depth
    min_samples_split=5,     # Minimum samples to split
    min_samples_leaf=2,      # Minimum samples in leaf
    ccp_alpha=0.01,          # Cost Complexity Pruning
    random_state=42
)
```